SUPERNOVA AGENTIC APPLICATION

The domain of the Project:

COURSE NAME

Generative AI

Team Mentors (and their designation):

Prujith Radhakrishna
GEN AI Mentor

Team Members:

Mr. Manoj Kumar

Period of the project

February 2026 to April 2026

Declaration

The project titled “SUPERNOVA AGENTIC APPLICATION” has been mentored by Prujith Radhakrishnan Sir, organized by SURE Trust, from February 2026 to April 2026, for the benefit of the educated unemployed rural youth for gaining hands-on experience in working on industry relevant projects that would take them closer to the prospective employer. I declare that to the best of my knowledge the members of the team mentioned below, have worked on it successfully and enhanced their practical knowledge in the domain.

Team Members:

Mr. Manoj Kumar

Mentor’s Name:

Prujith Radhakrishnan
GENERATIVE AI MENTOR — SURE TRUST

Executive Director & Founder
Prof. Radhakumari
SURE TRUST
SURE TRUST

Table of contents:

Executive Summary

SUPERNOVA Application is a personal AI desktop assistant built for Windows and designed to work across multiple user interfaces and control surfaces. The project combines a WPF desktop application, a Python FastAPI backend, a Telegram remote control bot, and an Android companion application into one connected product.

The main idea behind SUPERNOVA Application is simple: the user should be able to talk to the system in natural language and let it help with real computer tasks, routine actions, file access, remote monitoring, and mobile control. Instead of being only a chatbot, it is built as a practical assistant that can understand requests, route them to the correct service, and return useful results in a clear, user-friendly way.

The system is local-first and privacy-conscious. It is intended to run on the user's own machine, with remote access enabled only when the user chooses to connect through supported methods such as local network access, Telegram commands, or a tunnel-based setup. The architecture is modular so that the desktop interface, backend services, Telegram bot, and Android app can all grow independently.

At a high level, the product is designed for three major experiences:

- A polished Windows desktop interface for direct interaction and control.
- A Python backend that manages AI requests, tools, automation, and device state.
- Remote control through Telegram and Android so the assistant remains available even when the user is away from the PC.

Introduction

1.1 Background

Modern computer users often work across many environments at once. A person may be at a desktop PC, then move to a phone, then return to the computer later to continue the same task. Many routine activities, such as checking system status, finding files, starting applications, reading news, performing searches, or handling simple commands, still require manual interaction even though they could be automated.

At the same time, AI assistants have become more capable, but many of them are either too cloud dependent, too limited in control, or too disconnected from the real device where the work happens. That creates a gap between conversation and action.

1.2 Problem Statement

The problem addressed by SUPERNOVA Application is the lack of a unified assistant that can do all of the following in one system:

- Understand user intent in simple language.
- Control the Windows PC safely and reliably.
- Provide a strong desktop user interface for daily use.
- Offer remote access from a phone through Telegram and Android.
- Keep the architecture maintainable and modular for future growth.

1.3 Solution Overview

SUPERNOVA Application solves this by separating the product into clear layers. The WPF application gives the user a full desktop experience. The Python backend handles the logic, tools, safety checks, and AI communication. The Telegram bot gives quick command-based remote access. The Android application becomes the mobile companion for remote control, status checks, and interaction on the move.

This project is not just a utility script. It is a product platform. The value comes from connecting a desktop assistant, remote control, mobile access, automation services, and intelligent routing into one coordinated system.

Project Objectives

1.4 Primary Objectives

The primary objective of the project is to build a complete AI-powered personal assistant system for Windows that can support everyday productivity and remote access.

The core goals are:

1. Build a strong Windows desktop front end using WPF.
2. Create a reliable Python backend for AI, automation, and system control.
3. Enable remote usage through Telegram commands.
4. Build an Android companion app for mobile control and monitoring.
5. Keep the system modular so each layer can evolve separately.
6. Provide a simple and understandable user experience even when the system is technically advanced.

1.5 Functional Objectives

SUPERNOVA Application is designed to support practical tasks such as:

- Launching and controlling applications.
- Reading system health and device status.
- Sending screenshots or visual updates.
- Searching and working with files.
- Running AI-supported helper tasks.
- Supporting voice and chat interaction.
- Enabling remote device access from Telegram and Android.

1.6 Design Objectives

The project also aims to achieve the following design goals:

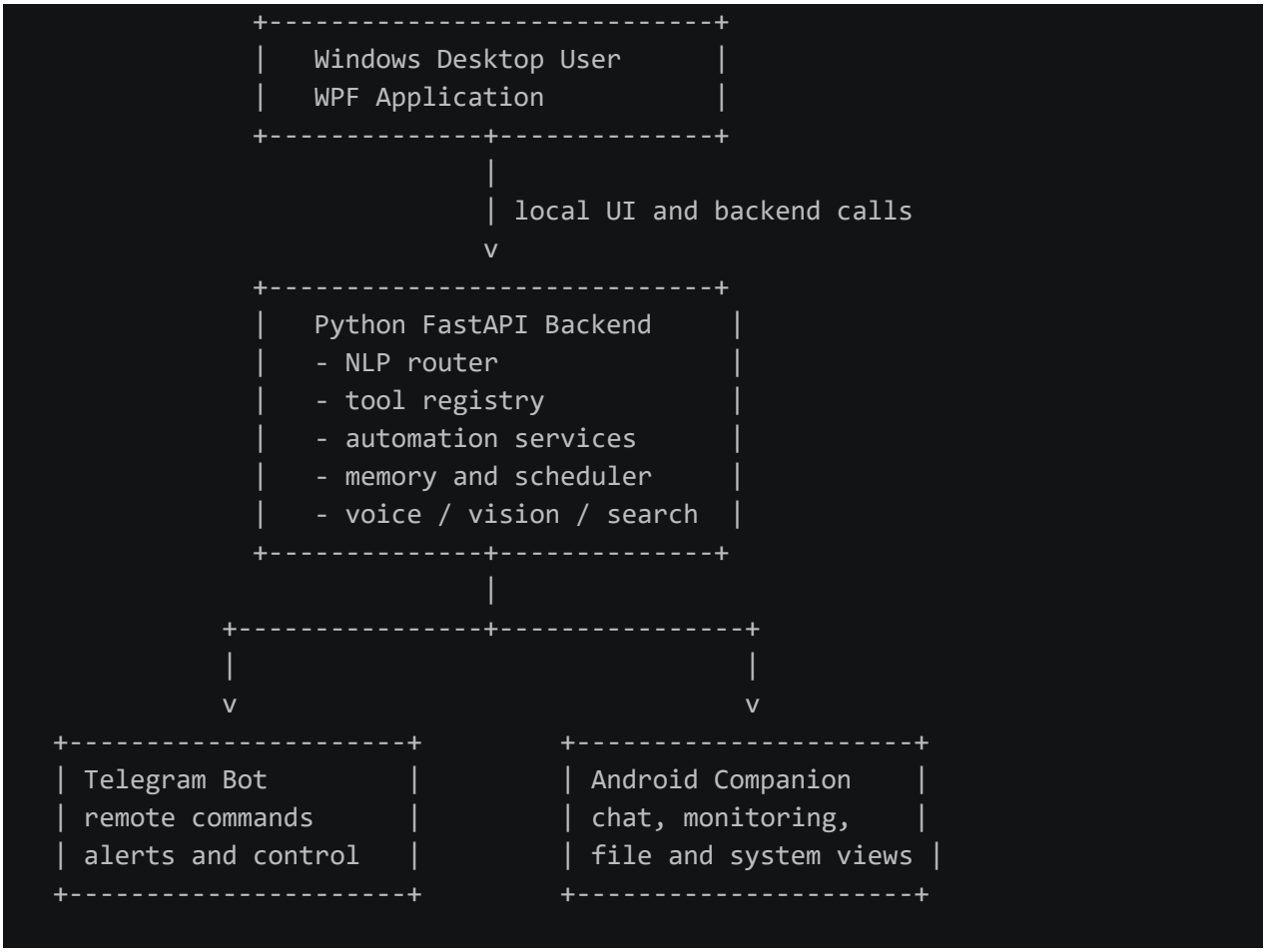
- Keep the desktop interface clean, modern, and easy to navigate.
- Make the backend stable and easy to maintain.
- Ensure the remote features are understandable for non-technical users.
- Allow future expansion without redesigning the entire product.

2. Methodology and system architecture

2.1 Overall Approach

The system follows a layered and service-oriented design. Each major capability is separated into its own part so that the project remains manageable as it grows. The UI, backend, remote bot, and Android app communicate through APIs, sockets, and service layers rather than depending on one giant block of code.

2.2 Architecture Summary



2.3 Core System Flow

The project follows a practical flow:

1. The user enters a command through WPF, Telegram, or Android.
2. The backend interprets the request and determines the best handler.
3. The appropriate service is called, such as system control, file handling, browser automation, vision,voice, or messaging.

4. The result is returned in a human-friendly format.

5. The UI or remote client shows the output and allows the user to continue the task.

2.4 Methodology Used

The development approach combines modular software design with iterative feature integration. The project is organized around service boundaries so that each domain can be developed, tested, and improved independently. This method is suitable for a product that includes desktop UI, backend APIs, mobile access, and bot-based remote control.

3. Technology Stack

3.1 Desktop Application (WPF)

- Framework: WPF (.NET)
- Language: C#
- Rendering: WebView2 for embedded web content
- UI Components: XAML-based controls with modern styling
- Key Libraries:
- - Microsoft.Extensions.DependencyInjection (DI container)
- - Newtonsoft.Json (JSON serialization)
- - ClientWebSocket (built-in for WebSocket communication)

3.2 Backend Services (Python)

Core Framework:

- FastAPI (HTTP API framework)
- Uvicorn (ASGI server running on port 8765)
- Pydantic (data validation and settings management)
- Python 3.x runtime

Database & Persistence:

- SQLite with WAL mode (Write-Ahead Logging)
- SQLAlchemy ORM for database operations
- aiosqlite for async database access

Communication & Remote Access:

- WebSocket for real-time streaming

- python-telegram-bot library for Telegram bot integration
- pyngrok for optional ngrok tunnel support
- FastAPI CORS middleware for cross-origin requests

3.3 System Control & Automation

Windows Automation:

- PyAutoGUI (keyboard and mouse control)
- PyGetWindow (window enumeration and management)
- PyWinAuto (advanced window automation)
- pyperclip (clipboard operations)
- pywin32 (Windows API access)
- pycaw (audio control)
- send2trash (safe file deletion)
- screen-brightness-control (brightness management)

Process & System Monitoring:

- psutil (CPU, RAM, disk, network monitoring)
- comtypes (COM interface access)
- APScheduler (task scheduling with SQLite job store)

3.4 Voice & Audio Processing

- Faster-Whisper (STT - Speech-to-Text)
- Piper TTS (Text-to-Speech synthesis) • Melo TTS (alternative TTS engine)
- PyAudio (audio input/output handling)
- Web Speech API support (for browser-based voice)

3.5 Vision & Screen Capture

- MSS (Multi-Screen Screenshot) for fast screen capture
- PaddleOCR (Optical Character Recognition)
- PIL/Pillow (image processing and manipulation)
- Vision models for element detection and interaction

3.6 Browser Automation

- Playwright (browser control and web automation)
- yt-dlp (YouTube video download and metadata)

- Jina Reader (web content extraction)
- Support for Chromium-based and Firefox browsers

3.7 AI & LLM Integration

LLM Providers Supported:

- Ollama (local models: qwen2.5:7b, qwen3:8b, gemma4:e4b)
- TypeGPT (cloud API with 5+ model options)
- OpenAI-compatible endpoint abstraction
- Vision model support (qwen3-vl:4b for image understanding)

LLM Routing:

- Deterministic command routing (60+ pattern matches)
- ReAct loop with LLM fallback (3 retries with self-correction)
- Auto/Local-only/Cloud-only routing modes

3.8 Security & Authentication

- JWT token generation for Android pairing (30-day expiration)
- QR code generation for pairing flows
- PIN lock support with hashing
- Environment variable support for sensitive credentials
- Request validation via Pydantic schemas

3.9 Data Formats & APIs

- JSON for all API communication
- YAML for configuration files
- QR code image generation (qrcode[pil])
- Multipart file upload/download
- MJPEG streaming for remote screen view

3.10 Development & Logging

- Logging to file with rotation support
- Comprehensive telemetry tracking
- Event-driven architecture with event bus
- Type hints throughout codebase (Python 3.9+)

4. Features & Capabilities

4.1 Windows Desktop Experience

The WPF application is the primary desktop interface for SUPERNOVA Application. It provides a modern, responsive user experience for real-time interaction with the assistant.

Core Chat Interface:

- Real-time WebSocket streaming of AI responses (token-by-token)
- Message history with persistent storage
- Syntax highlighting for code blocks
- Markdown rendering for formatted responses
- User input box with auto-suggestion
- Message timestamps and sender identification

Status & Connection Indicators:

- Real-time connection status pill (Connected/Connecting/Offline)
- Python backend auto-launch (PythonLauncher.cs)
- Backend health check on startup
- Port availability detection (8765)
- Graceful fallback if backend already running

Voice Features:

Floating voice pill window (`voice.html`) with live waveform

Voice hotkey support (Ctrl+Shift+Space by default)

Real-time transcript display

- Voice input in multiple languages
- Voice output (TTS) with multiple voices
- Stop speaking button

System Tray Integration:

- Minimize to system tray
- Right-click context menu
- Quick access window (Win+Space)
- Tray icon with hover tooltip

UI Themes:

- Dark theme (default, cosmic styling)
- Light theme
- Faded theme
- Shaded theme
- Smooth transition animations between themes
- Persistent theme preference

Settings Panel:

- LLM provider selection (Ollama, TypeGPT)
- Model selection and discovery
- API key management (TypeGPT, INFIP)
- Voice and TTS engine configuration
- Vision model selection
- Browser automation settings
- Theme preferences
- Hotkey customization
- Remote access configuration
- Telegram bot setup
- PIN lock for security

Loading & Startup:

- Animated supernova SVG logo during startup
- Progress bar with loading percentage
- Full-UI skeleton loading (shimmer effect)
- Expected startup time: 5-7 seconds
- Graceful error messaging

Backend Integration:

- WebSocket connection to Python backend (ws://127.0.0.1:8765/ws)
- Session management
- Message serialization/deserialization
- Automatic reconnection on disconnects
- Streaming response handling

4.2 Python Backend

The backend is the core engine of the project. It interprets user requests, coordinates service calls, and exposes the APIs used by the desktop app, Telegram bot, and Android client.

REST API Endpoints:

- `/api/health` - System health and metrics
- `/api/commands/*` - Command execution and routing
- `/api/voice/*` - STT, TTS, and voice settings
- `/api/screen/*` - Screenshot, click, type, key events
- `/api/files/*` - File search, upload, download, operations
- `/api/memory/*` - Store and retrieve facts
- `/api/tasks/*` - Task scheduling and management
- `/api/settings/*` - Configuration and Android pairing

WebSocket Streaming:

- Real-time token streaming for LLM responses
- Live metrics (CPU, RAM, disk, battery)
- Error notifications
- Task status updates

NLP & Command Routing:

- 60+ deterministic pattern matches for common commands
- LLM-based fallback for complex requests
- ReAct loop with 3 retry attempts and self-correction
- Multi-intent detection and disambiguation

Tool Execution Groups (13 total):

- System Control: Power, sleep, lock, restart, brightness, volume
- Application Management: Launch, close, switch, enumerate windows
- File Operations: Create, read, delete, rename, move, search, zip/unzip
- Vision & OCR: Screenshot, element detection, OCR reading, find-and-click
- Browser Automation: Open URLs, search, form filling, content scraping
- Audio Control: Media playback, volume, mute/unmute
- Terminal Safety: PowerShell with LLM safety validation and command allowlisting
- Memory Management: Store facts, retrieve context, SQLite persistence
- Web Intelligence: Web search (Jina), news briefing, URL content extraction
- Content Creation: Image generation, document formatting, code snippets
- Scheduling: APScheduler-backed task scheduling with persistence
- Messaging: Telegram send, WhatsApp webhook (stub)
- Network: Network info retrieval and control

Voice & Audio:

- Faster-Whisper STT with large-v3-turbo model
- Piper and Melo TTS engines
- Audio level monitoring
- Wake word simulation

Vision & Perception:

- MSS-based screenshot capture at configurable resolution
- PaddleOCR for text extraction
- Vision model integration (qwen3-vl:4b)
- Element confidence scoring and click strategy selection

Memory & Context:

- SQLite-backed fact storage
- Conversation history persistence
- User preference caching

Scheduling & Automation:

- APScheduler with SQLite job store
- Persistent task scheduling across restarts
- Optional Telegram notifications on task completion
- Task listing, status tracking, and cancellation

4.3 Telegram Remote Control

The Telegram bot provides a complete remote control interface through messaging. Users can send commands from any device with Telegram installed.

Implemented Commands:

- `/start` - Bot introduction and inline menu
- `/help` - Command reference
- `/screenshot` or `/screen` - Capture and send desktop screenshot
- `/stats` - System statistics (CPU, RAM, disk, uptime, battery)
- `/ps` - Process list enumeration
- `/kill <process>` - Terminate process by name
- `/privacy` - Toggle privacy mode (blur wallpaper)
- `/fetch <URL>` - Web content fetching and summarization
- `/mute` - System audio mute
- `/unmute` - System audio unmute
- `/volume <level>` - Set system volume (0-100)
- `/sleep` - Put system to sleep
- `/lock` - Lock Windows session
- `/restart` - Restart system
- `/open <app>` - Launch application
- `/close <app>` - Close application
- `/type <text>` - Type text on screen
- `/click <x> <y>` - Mouse click at coordinates
- `/memory <fact>` - Store a fact
- `/forget <topic>` - Forget stored facts
- `/models` - List available LLM models
- `/setmodel <model>` - Change active AI model
- `/stream <on|off>` - Enable/disable token streaming

Advanced Features:

- Voice message transcription â†' AI reply
- Document upload â†' save to PC
- Inline keyboard menu for quick actions
- Rate limiting per user (prevent abuse)
- Duplicate update deduplication
- Resilient retry with exponential backoff
- User ID authorization (single-user security)
- Command confirmation dialogs for dangerous operations
- Telemetry and usage tracking per command

Status Monitoring:

- Real-time system health checks
- Battery level reporting

  Uptime tracking

  Network status and connectivity info

4.4 Android Companion App

The Android application (Flutter-based) extends SUPERNOVA Application to mobile devices. It provides a dedicated mobile interface for remote interaction, status monitoring, and control when away from the desktop.

Available APIs for Android:

- `GET /api/connection-info` - Fetch local and tunnel URLs
- `GET /api/connection-info/qr` - QR code PNG for pairing
- `POST /api/auth/generate-token` - Generate 30-day JWT token
- `GET /api/system/screen/snapshot` - JPEG screenshot of desktop
- `POST /api/system/screen/click` - Remote mouse click at coordinates
- `POST /api/system/screen/type` - Remote keyboard text input
- `POST /api/system/screen/key` - Hotkey execution (e.g., Ctrl+C)
- `GET /api/files/search?query=` - Search files on PC
- `GET /api/files/download?path=` - Download file to phone
- `POST /api/files/upload` - Upload file from phone to PC
- `GET /api/system/health` - System metrics and status
- `GET /api/system/screen/stream` - MJPEG live desktop stream

Planned Android Features:

- QR code scanning for pairing
- Manual connection entry (IP + token)
- Glassmorphic UI design with cosmic theme
- Real-time chat interface with WebSocket streaming
- Remote desktop view with touch-to-click interaction
- File browser and transfer UI
- System dashboard with CPU/RAM/disk gauges
- Connection status indicator
- Settings panel (disconnect, regenerate token, FPS slider)
- Voice command input
- Push notifications for alerts (Firebase Cloud Messaging ready)

Architecture (Flutter):

- State management: Provider (ChangeNotifier)
- Networking: dio + http + web_socket_channel
- Storage: flutter_secure_storage (encrypted JWT), shared_preferences

  QR Scanning: mobile_scanner

  File Access: file_picker + path_provider
- JWT persistence across app sessions
- Automatic reconnection with exponential backoff

4.5 Major Functional Areas

SUPERNOVA Application is designed around the following functional domains, each with dedicated backend services:

System & Device Control:

- CPU, RAM, disk, battery monitoring
- Volume and brightness adjustment
- System power states (sleep, lock, restart, shutdown)
- Process enumeration and termination
- Network information retrieval
- Windows session management

AI-Based Command Understanding:

- Deterministic routing for known commands (60+ patterns)
- LLM-based intent classification for unknown requests
- ReAct loop for complex multi-step tasks
- Self-correction with retry mechanism
- Context awareness from conversation history

Voice Interaction:

- Speech-to-text (Faster-Whisper)
- Text-to-speech (Piper or Melo)
- Wake word simulation
- Voice settings persistence
- Language support configuration

Vision & Screen Capture:

- Full-screen or region screenshots
- OCR text extraction from images
- Element detection and localization
- Find-and-click workflows
- Vision model inference (qwen3-vl:4b)

File Management:

Recursive file search with query matching

File creation, deletion, rename, move operations

- Archive operations (zip/unzip)
- File upload/download via REST API
- Safe deletion using send2trash

Scheduled Tasks:

- APScheduler-backed job scheduling
- Persistent SQLite job store
- Task status tracking
- Task cancellation
- Optional Telegram notifications

- Recurring task support

Remote Connectivity:

- ngrok tunnel support for internet access
- Local network access (LAN)
- JWT token-based authentication
- QR code pairing for Android
- Session isolation per client

Telegram Command Execution:

- All system tools accessible via Telegram
- Rate limiting per user
- Confirmation dialogs for destructive operations
- Inline keyboard menus
- Message editing for real-time updates

Android Companion Access:

- Screenshot streaming (MJPEG)
- Remote control (click, type, keys)
- File transfer workflows
- System health monitoring
- Real-time metrics via WebSocket

Content Creation & Automation:

- Image generation via INFIP API
- Document creation
- Code snippet generation

  Web content fetching and summarization

  YouTube video metadata extraction (yt-dlp)

Memory & Persistence:

- SQLite fact storage
- Conversation history archival
- User preference caching
- Configuration YAML-based storage
- Event-driven architecture with event bus

4.6 User Value & Real-World Applications

Productivity Improvements:

- Reduce manual task switching between PC and phone
- Execute repetitive commands via natural language
- Monitor system health from anywhere
- Schedule tasks to run while away
- Hands-free voice control

Use Cases:

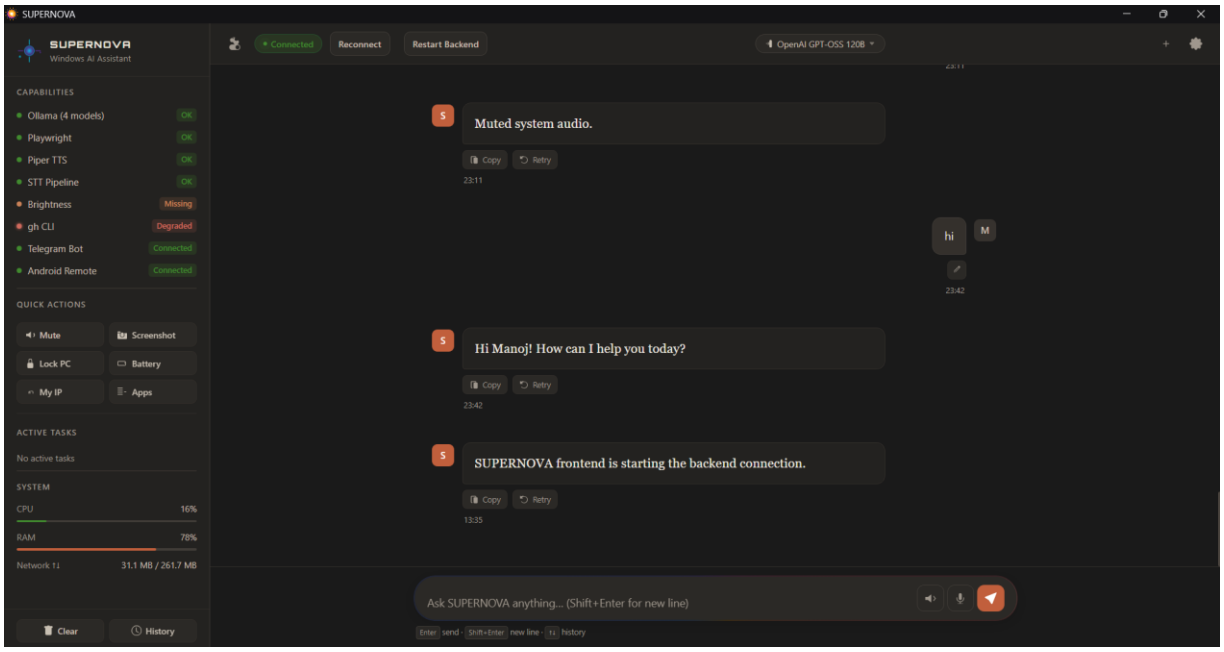
- Remote PC monitoring and control while away from desk
- Voice-driven task execution ("take a screenshot", "open Chrome", "check CPU")
- File transfer between PC and phone seamlessly
- Scheduling important reminders and actions
- Voice diary or note-taking with AI summarization
- Automated workflow execution (e.g., download file â†' process â†' upload)
- Multi-device productivity (chat on phone, execute on PC)
- Accessibility support for users with mobility limitations

Business Applications:

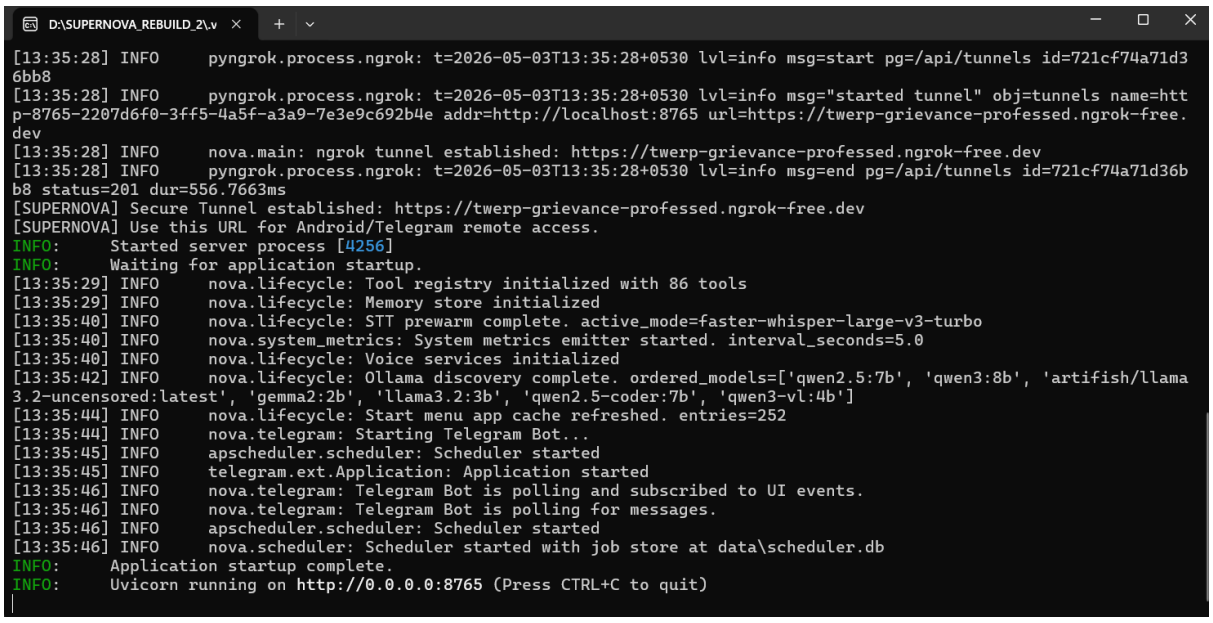
- Personal automation for knowledge workers
- Remote system administration
- Task scheduling for batch operations
- System health monitoring for small office PCs
- Cross-device workflow management

5. Screenshots

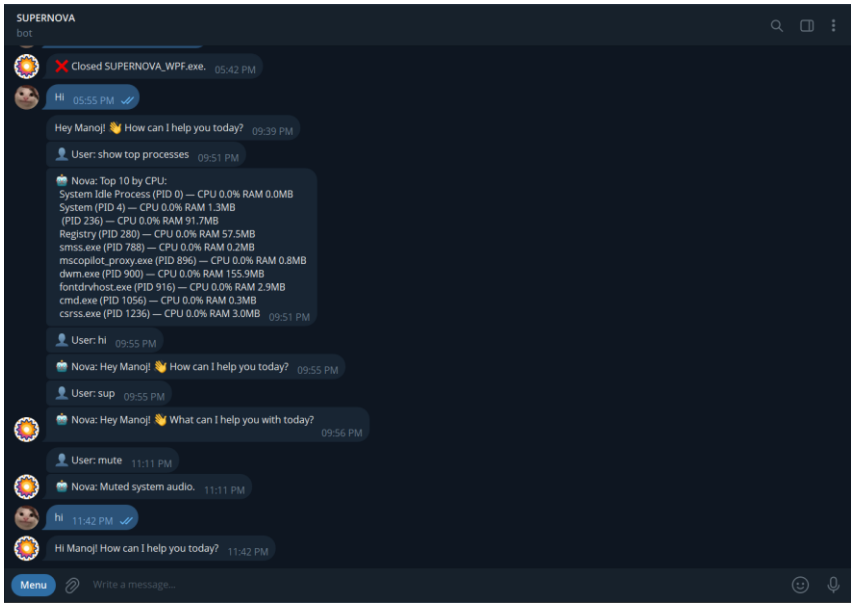
5.1 Desktop Application Screenshot



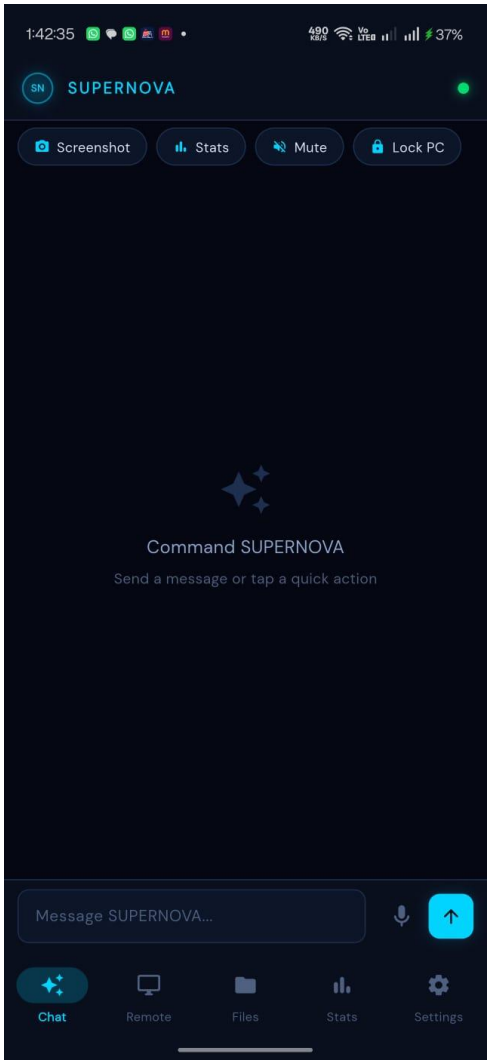
5.2 Backend or Service Screen



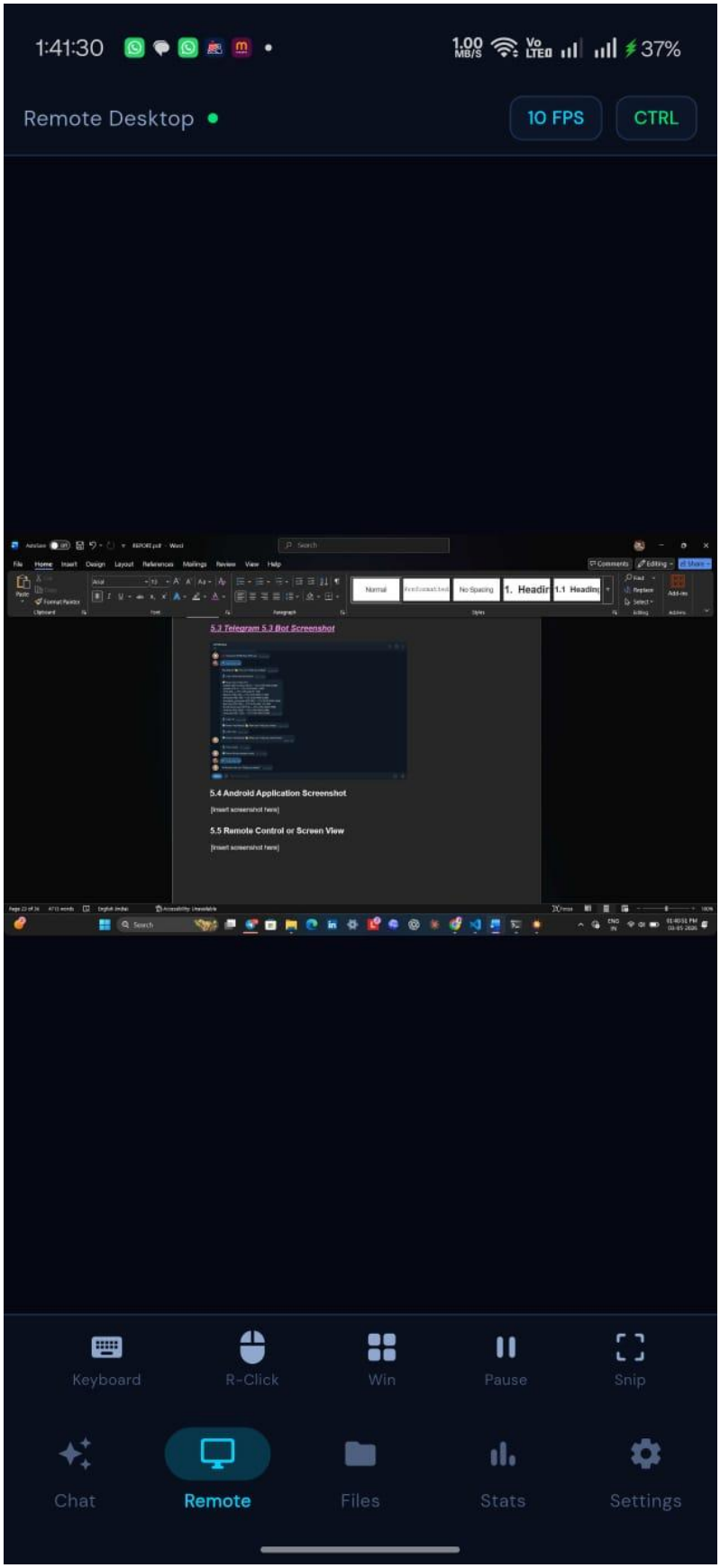
5.3 Telegram 5.3 Bot Screenshot



5.4 Android Application Screenshot



5.5 Remote Control or Screen View



6. Installation and Deployment

6.1 System Requirements

Hardware:

- Windows 10/11 64-bit or later
- 8 GB RAM minimum (16 GB recommended for Ollama models)
- 10 GB free disk space (20+ GB if running local LLMs)
- Intel/AMD processor with SSE4.2 support
- Optional: Dedicated GPU (NVIDIA CUDA, AMD ROCm) for faster inference

Software Prerequisites:

- Python 3.9+ (backend)
- .NET 6/7 (WPF desktop app)
- Visual Studio 2022 or later
- Flutter SDK (for Android app)
- Ollama (optional, for local LLMs ~7-8 GB per model)

6.2 Backend Installation (Python)

1. Clone and navigate to repository

```
git clone <repository-url>
cd d:\SUPERNOVA_REBUILD_2
```

2. Create Python virtual environment

```
python -m venv .venv .
\venv\Scripts\activate
```

3. Install dependencies

```
pip install --upgrade pip pip
install -r requirements/base.txt
```

4. Optional: Melo TTS support (requires Python 3.10)

```
python3.10 -m venv. venv310
.\venv310\Scripts\activate pip
install -r requirements/melo.txt
deactivate
```

Set environment variable:
`NOVA_MELO_PYTHON=\.venv310\Scripts\python.exe`

5. Install Ollama (optional, for local LLMs)

```
# Download from https://ollama.ai ollama serve  # Start
service (runs on http://127.0.0.1:11434) ollama pull
qwen2.5:7b ollama pull qwen3:8b
ollama pull qwen3-vl:4b    # Vision model
```

6. Configure settings

```
cp config.yaml.example config.yaml
# Edit config.yaml with:
# - API keys (TypeGPT, INFIP) - leave blank if using only Ollama
# - Telegram bot token and user ID (optional)
# - ngrok token (optional, for remote access)
```

7. Start the backend

```
.\venv\Scripts\activate python main.py #
Backend will start on http://127.0.0.1:8765
```

Or use PowerShell script:

```
.\start_backend.ps1
```

Verification:

```
curl http://127.0.0.1:8765/health # Should
return: {"status": "healthy", ...}
```

6.3 Desktop Application (WPF)

1. Open project in Visual Studio

```
cd NOVA_WPF Start SUPERNOVA_WPF.csproj  # Opens in
Visual Studio
```

2. Ensure backend is running

- Backend must be running on port 8765 before launching desktop app

  Desktop app will show connection error if backend is unavailable

3. Build and run

- Visual Studio: Press F5 or Build â† Run
- Command line:

```
dotnet restore
dotnet build
dotnet run
```

Quick Launch Alternative:

```
Application_exe.bat
```

This batch file:

1. Checks if backend is running
2. Starts backend if not running
3. Launches WPF desktop app
4. Auto-restarts on crash

6.4 Android Companion App Setup

Prerequisites:

- Flutter SDK installed
- Android Studio or Android SDK tools
- Nexus/Android emulator or physical device

Steps:

1. Ensure backend has Android pairing enabled

```
 In config.yaml android_pairing_enabled:
true
```

2. Build Flutter app

```
cd Android\flutter\supernova_remote
flutter pub get flutter build apk -
-release
```

3. Deploy to device

```
flutter install # Or manually: adb install
build\app\outputs\apk\release\app-release.apk
```

4. Pairing Flow

- Open Android app
- Go to Settings â†’ "Connect to Desktop"
- Choose "Scan QR Code" or "Manual Connection"
- On desktop: Settings â†’ Android â†’ "Generate Pairing QR"
- Scan with phone â†’ Connection established JWT token automatically generated and stored

6.5 Telegram Bot Setup

1. Create Telegram bot

- Open Telegram, find @BotFather
- Send `/newbot`
- Follow prompts to name bot and get token
- Copy the HTTP API token

2. Find your Telegram user ID

- Search for @userinfobot in Telegram
- Send any message
- Bot replies with your user ID (e.g., 123456789)

3. Configure SUPERNOVA

```
# In config.yaml
server:
  telegram_enabled: true
telegram_bot_token: "YOUR_TOKEN_HERE"
telegram_user_id: "YOUR_USER_ID_HERE"
```

4. Restart backend

```
Backend automatically registers all bot commands on startup
Bot becomes available for interaction immediately
```

5. Test in Telegram

- Search for your bot (e.g., @YourBotName)
- Send `/start` should show menu
- Send `/stats` should return system metrics

6.6 Remote Access via ngrok (Optional)

For access outside your local network:

1. Get ngrok account:

- Sign up free at https://ngrok.com
- Copy auth token from dashboard

2. Configure backend

In config.yaml

```
server:
remote_enabled: true
ngrok_enablngrok_token:
"YOUR_NGROK_TOKEN_HERE"
```

3. Backend auto-creates tunnel:

- On startup, backend logs ngrok URL
- Example: `INFO - ngrok tunnel: https://abcd-1234-efgh.ngrok.io`
- Share URL with Android app or remote Telegram user

4. Configure Android/clients:

- Copy tunnel URL from backend logs
- In Android app: Settings Server URL paste tunnel URL
- Automatic reconnection on network change

6.7 Deployment Verification

Backend API test:

Health check

```
curl http://127.0.0.1:8765/health
```

Command execution

```
curl -X POST http://127.0.0.1:8765/api/commands/execute \
  -H "Content-Type: application/json" \
  -d '{"command": "show system info"}'
```

Voice settings

```
curl http://127.0.0.1:8765/api/voice/settings
```

Desktop app test:

1. Launch Application_exe.bat
2. Wait 5-7 seconds for startup
3. Check connection status indicator
4. Type a message and verify response
5. Press Ctrl+Shift+Space to test voice activation

Telegram test:

1. Send `/help should show command list
2. Send `/screenshot` should return desktop screenshot
3. Send `/stats` should return CPU/RAM/disk/battery

Android test:

1. Open app and wait for connection
2. Type message in chat and verify response
3. Tap "Snapshots" to view live desktop MJPEG stream
4. Tap "Control" and try clicking on stream

6.8 Troubleshooting

Backend won't Start?

- Port 8765 already in use: `netstat -ano | findstr 8765`
- Kill process: `taskkill /PID <PID> /F`
- Missing dependencies: `pip install -r requirements/base.txt`
- Python version: Ensure Python 3.9+
- Check logs: `type logs\nova.log`

Desktop app connection fails?

- Verify backend is running: `curl http://127.0.0.1:8765/health`
- Check firewall: Windows Defender may block Uvicorn port 8765
- Restart both backend and desktop app
- Check logs: Desktop app logs to `logs\` directory

Ollama models not discovered?

- Ensure Ollama service is running: `ollama serve`
- Test connection: `curl http://127.0.0.1:11434/api/tags`
- Pull model: `ollama pull qwen2.5:7b`
- Check config.yaml `ollama.base_url` setting

Telegram bot not responding?

- Verify token in config.yaml (no typos)
- Confirm user ID matches your Telegram ID (test with @userinfobot)
- Restart backend: `python main.py`
- Check logs: `type logs\telegram.log`

Android pairing fails?

- Backend: Ensure `android_pairing_enabled: true` in config.yaml
- Desktop: Go to Settings â†' Android â†' Generate Token â†' Show QR
- Network: Both PC and phone must be on same WiFi or via ngrok tunnel
- Firewall: Allow port 8765 in Windows Defender
- ngrok: If using remote access, ensure ngrok tunnel URL is in Android settings

Voice activation not working?

- Check hotkey setting: config.yaml â† `voice_hotkey` (default: `ctrl+shift+space`)
- Verify mic permission: Windows Settings â† Privacy & Security â† Microphone
- Test mic: Settings â† Sound â† Volume mixer â† recording device
- Check logs: `type logs\nova.log` for voice errors

Vision/OCR not working?

- Ensure Ollama vision model: `ollama pull qwen3-vl:4b`
- PaddleOCR downloads on first use (may take time)
- Check disk space for model caches (~2 GB)
- Verify GPU if using CUDA acceleration

7. Result

7.1 Project Outcome

The final outcome of SUPERNOVA Application is a multi-surface assistant platform that combines local desktop computing with remote access through Telegram and Android. This makes the system more practical than a single-interface assistant because it supports usage from both the PC and mobile devices.

7.2 Observed Benefits

- Faster access to common actions.
- Better control of the desktop system.
- Easy remote interaction from a phone.
- A clear base for future feature growth.
- Improved user convenience for daily tasks.

7.3 Technical Results

The project demonstrates that a local-first assistant can be designed as a modular platform rather than as a single script or narrow chatbot. The architecture supports different interfaces, different control styles, and different user contexts while keeping the core logic in the backend.

8. Social And Industry Relavance

8.1 Social Relevance

SUPERNOVA Application is useful for people who want easier computer access, more direct remote control, and simpler interaction with a personal machine. It can be especially helpful for users who prefer speaking or chatting instead of manually navigating through many screens.

The system also supports convenience for users who are away from their desktop but still need to monitor or control it.

8.2 Industry Relevance

The project reflects a practical industry direction where AI is moving beyond chat and into action. The combination of automation, remote access, desktop integration, and mobile support mirrors real product patterns seen in modern AI-assisted systems.

The architecture is relevant to:

- Personal productivity tools.
- Remote device management.
- AI-assisted desktop software.
- Companion mobile applications.
- Hybrid local and remote-control systems.

8.3 Wider Impact

This type of system can help show how AI can be applied to everyday computing in a concrete way. Instead of staying as a demonstration, the assistant becomes something useful for real work, daily operations, and user support.

9. Learning And Reflection

9.1 Technical Learning

This project provides experience in several important areas:

- Designing a modular application architecture.
- Building a desktop application for Windows.
- Creating backend services in Python.
- Connecting multiple clients to one core system.
- Handling remote commands and device control.
- Planning mobile companion app workflows.

9.2 Practical Reflection

The most important lesson from the project is that useful AI software must be more than model integration. It must also be organized, reliable, and connected to the real environment where work happens. A strong assistant needs a strong backend, a clear interface, and a safe way to execute actions.

9.3 Personal Growth

Working on SUPERNOVA Application improves understanding of software product design, integration thinking, and user-focused development. It also shows how to combine desktop, backend, bot, and mobile layers into one project with a shared purpose.

10. Future Scope

10.1 Planned Enhancements

The project already has a strong base, and the next stages can expand it further. Possible future work includes:

- Full public release of the Windows `.exe` package.
- Full public release of the Android APK.
- Improved remote control experience on mobile.
- More advanced task automation.
- Additional safety and access controls.
- Better notifications and status updates.
- More refined synchronization between desktop and phone.

10.2 Product Growth Direction

Future versions can extend the assistant into a broader ecosystem that supports more device types, richer dashboards, and deeper task handling. Because the system is modular, each part can be improved without rebuilding the entire product.

11. Conclusion

SUPERNOVA Application is a complete personal AI assistant project that brings together a Windows desktop app, a Python backend, Telegram remote control, and an Android companion app. The project is designed to make computing more accessible, more mobile, and more conversational.

The main strength of the system is that it connects real action with simple user interaction. It does not stay at the level of ideas or chat only. It is meant to work as a practical product that can help users manage their machine, stay connected remotely, and use AI in a more useful everyday form.

12. References

1. FastAPI Documentation
2. Uvicorn Documentation
3. WPF and .NET Documentation
4. Flutter and Dart Documentation
5. Telegram Bot API Documentation
6. Python Documentation
7. SQLite Documentation
8. APScheduler Documentation
9. Internal project files and implementation notes in this repository